भारतीय सूचना प्रौद्योगिकी संस्थान गुवाहाटी

Indian Institute of Information Technology Guwahati

CS236 - Artificial Intelligence Lab

Assignment - 12

Instructions

Implement Prolog predicates for the following problems. Ensure your code is well-commented and follows good Prolog style. Test your predicates with the provided examples and consider edge cases where appropriate.

Part 1: Prolog Basics & Arithmetic

1. Explain the difference between the Prolog operators `=`, `is`, and `=:=`. Provide example queries for each operator that demonstrate their distinct behavior, including cases where they succeed and fail, especially involving arithmetic expressions and variables. For example, consider queries like:

   - `+(3, 5) = 3 + 5.`
   - `+(3, 5) is 3 + 5.`
   - `+(3, 5) =:= 3 + 5.`
   - `3 + 5 is +(3, 5).`
   - `3 + 5 = +(3, 5).`
   - `3 + 5 =:= +(3, 5).`
   - `X = 3 + 5.`
   - `X is 3 + 5.`
   - `8 = 3 + 5.`
   - `8 is 3 + 5.`
   - `3 + 5 =:= 8.`
   - `3 + 5 = 8.`
   - `X is 3 + 5, X =:= 8.`
   - `X = Y.`
   - `X is Y.` (What happens here?)

   Document the results and explain why Prolog responds the way it does for each query based on the operator's definition (matching, arithmetic evaluation, arithmetic comparison).

2. Write a Prolog predicate `max(X, Y, Max)` that takes two numbers `X` and `Y` and unifies `Max` with the larger of the two.

```
?- max(10, 5, M).
M = 10
Yes
?- max(7, 7, M).
M = 7
Yes
```

3. Write a Prolog predicate `is_leap_year(Year)` that succeeds if `Year` is a leap year and fails otherwise. Recall the rules: A year is a leap year if it is divisible by 4, unless it is divisible by 100 but not by 400. You will need the modulo operator (`mod`).

```
?- is_leap_year(2000).
Yes
?- is_leap_year(2024).
Yes
?- is_leap_year(1900).
No
?- is_leap_year(2023).
No
```

4. Write a Prolog predicate `is_prime(N)` that succeeds if `N` is a prime number (an integer greater than 1 with only 1 and itself as divisors) and fails otherwise. You might want to implement a helper predicate `has_divisor(N, I)` that checks if `N` has a divisor between `I` and `sqrt(N)`. Use cuts (!) appropriately to make the check efficient once a divisor is found.

```
?- is_prime(17).
Yes
?- is_prime(18).
No
?- is_prime(2).
Yes
?- is_prime(1).
No
```

Part 2: List Manipulation

5. Write a Prolog predicate `analyse_list/1` that takes a single argument.

   - If the argument is a non-empty list, it should print the head and the tail of the list on separate lines, followed by succeeding.
   - If the argument is an empty list (`[]`), it should print a message indicating that the list is empty, followed by succeeding.
   - If the argument is not a list at all, the predicate should simply fail.

   You will likely need type-checking predicates like `is_list/1` (if available in your Prolog system; otherwise, you might need to simulate the check based on structure `[]`/`[H|T]`) and potentially checks for the empty list. Use `write/1` and `nl/0` for output.

   Examples:

```
?- analyse_list([dog, cat, horse, cow]).
Head: dog
Tail: [cat, horse, cow]
Yes

?- analyse_list([]).
This is an empty list.
Yes

?- analyse_list(just_an_atom).
No

?- analyse_list([only_head]).
Head: only_head
Tail: []
Yes
```

6. Write a Prolog predicate `membership/2` that works like the built-in `member/2` (without using `member/2`). Use recursion and the head/tail pattern.

7. Write a Prolog predicate `mylength(List, Length)` that calculates the number of elements in `List` and unifies the result with `Length`. Do not use the built-in `length/2`.

```
?- mylength([a, b, c], L).
L = 3
Yes
?- mylength([], L).
L = 0
Yes
```

8. Implement a predicate for returning the last element of a list in two ways:

   i. `last1(List, Element)` using recursion and the head/tail pattern.

   ii. `last2(List, Element)` using `append/3` without explicit recursion in `last2/2`.

9. Write a Prolog predicate `replace(ListIn, Old, New, ListOut)` that takes an input list (`ListIn` ↪), an element to be replaced (`Old`), and a replacement element (`New`). It should unify `ListOut` with a new list where every occurrence of `Old` in `ListIn` has been replaced by `New` ↪ . Other elements should remain unchanged. This will likely involve recursion.

   Example:

```
?- replace([1, 2, 3, 4, 3, 5, 6, 3], 3, x, List).
List = [1, 2, x, 4, x, 5, 6, x]
Yes

?- replace([a, b, c], d, e, Result).
Result = [a, b, c]
Yes

?- replace([], 1, z, Result).
```

```
Result = []
Yes
```

10. Write a predicate `palindrome(Atom)` that succeeds if the given Prolog `Atom` reads the same forwards and backwards. You will likely need built-in predicates to convert the atom to a list of characters (e.g., `atom_chars/2`) and potentially a list reversal predicate (you can use the built-in `reverse/2` or implement your own).

```
?- palindrome(madam).
Yes
?- palindrome(racecar).
Yes
?- palindrome(hello).
No
```

11. Write a Prolog predicate `element_at(List, N, Element)`.

    - `List` is the input list.
    - `N` is a positive integer representing the 1-based index (the first element is at index 1, the second at index 2, and so on).
    - `Element` should be unified with the element found at the `N`th position in the `List`.
    - The implementation should likely use recursion. Consider the base case: what is the first element (`N=1`) of a list `[Head|Tail]`?
    - For the recursive step (`N>1`), how does finding the Nth element relate to finding an element in the tail of the list? You will need to decrement the index.
    - If `N` is less than 1, or greater than the length of the list, or if the first argument is not a list, the predicate should fail.

    Examples:

```
?- element_at([tiger, dog, teddy_bear, horse, cow], 3, X).
X = teddy_bear
Yes

?- element_at([a, b, c, d], 1, Elem).
Elem = a
Yes

?- element_at([a, b, c, d], 4, Elem).
Elem = d
Yes

?- element_at([a, b, c, d], 5, X). % Index out of bounds (too large)
No

?- element_at([a, b, c, d], 0, X). % Index out of bounds (too small)
No
```

```
?- element_at([], 1, X). % Index out of bounds (empty list)
No
```

12. Write a predicate `split_list(Threshold, ListIn, LessList, GreaterEqList)` that splits `ListIn` ↪ (containing numbers) into two lists: `LessList` containing elements less than `Threshold`, and `GreaterEqList` containing elements greater than or equal to `Threshold`. Maintain the relative order of elements within the sublists.

```
?- split_list(5, [3, 8, 1, 5, 9, 4], Less, GE).
Less = [3, 1, 4],
GE = [8, 5, 9]
Yes
?- split_list(10, [2, 4, 6], Less, GE).
Less = [2, 4, 6],
GE = []
Yes
```

Part 3: Recursion, Databases, and Logic

13. Write a Prolog predicate `fibonacci(N, F)` to compute the Nth Fibonacci number `F`.

    - `N` is a non-negative integer input (0, 1, 2, ...).
    - `F` should be unified with the Nth Fibonacci number.
    - Use the standard recursive definition:
      - $F_0 = 0$ (Base case 1)
      - $F_1 = 1$ (Base case 2)
      - $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$ (Recursive step)
    - Your implementation should directly translate this definition into Prolog rules. You will need arithmetic evaluation (`is`) and comparison (`>=`, `=:=`).
    - Ensure your predicate correctly handles the two base cases (`N=0` and `N=1`) and the recursive step for `N >= 2`.

    Examples:

```
?- fibonacci(0, F).
F = 0
Yes

?- fibonacci(1, F).
F = 1
Yes

?- fibonacci(2, F).
F = 1
Yes

?- fibonacci(3, F).
```

```
F = 2
Yes

?- fibonacci(7, F).
F = 13
Yes

?- fibonacci(10, F).
F = 55
Yes
```

14. Write a predicate `mysin(X, N, Result)` that calculates an approximation of $sin(X)$ (where X is in radians) using the first `N` terms of its Taylor series expansion: $sin(X) = X - \frac{X^3}{3!} + \frac{X^5}{5!} - \frac{X^7}{7!} + \cdots = \sum_{k=0}^{N-1}(-1)^k \frac{X^{2k+1}}{(2k+1)!}$ You will need helper predicates for factorial and exponentiation (power/3). Calculate the sum for k from 0 up to N-1.

```
% Example: Calculate sin(pi/2) using first 3 terms (N=3, so k=0, 1, 2)
?- Pi is pi, X is Pi / 2, mysin(X, 3, R).
R = 0.9998... % Approximation should be close to 1.0
Yes
```

15. You are given a database consisting of facts defining basic family relationships:

    - `male(Person)`: True if `Person` is male.
    - `female(Person)`: True if `Person` is female.
    - `parent(Parent, Child)`: True if `Parent` is a parent of `Child`.

    Database Facts:

```
female(mary).
female(sandra).
female(juliet).
female(lisa).
male(peter).
male(paul).
male(dick).
male(bob).
male(harry).

parent(bob, lisa).
parent(bob, paul).
parent(bob, mary).
parent(juliet, lisa).
parent(juliet, paul).
parent(juliet, mary).
parent(peter, harry).
parent(lisa, harry).
parent(mary, dick).
parent(mary, sandra).
```

Based **only** on these facts, define Prolog **rules** for the following more complex family relations. You may need to combine several conditions using conjunction (‘,‘). You might also find the ”not equal” operator `X = Y` useful, which succeeds if `X` and `Y` cannot be unified (i.e., they are distinct).

i. `father(Father, Child)`: Should be true if `Father` is the father of `Child`. (Hint: A father is a male parent).

```
?- father(bob, lisa).
Yes
?- father(juliet, paul).
No % Juliet is female
?- father(X, harry).
X = peter
Yes
```

ii. `sister(Sister, Person)`: Should be true if `Sister` is the sister of `Person`. (Hint: Sisters share at least one parent, are female, and are not the same person).

```
?- sister(lisa, paul).
Yes
?- sister(mary, sandra). % Half-sister in this data, still counts
Yes
?- sister(lisa, lisa). % Should fail: Cannot be own sister
No
?- sister(paul, lisa). % Paul is male
No
?- sister(X, dick).
X = sandra
Yes
```

iii. `grandmother(GMother, GrandChild)`: Should be true if `GMother` is the grandmother of `GrandChild`. (Hint: A grandmother is a female parent of a parent).

```
?- grandmother(juliet, harry).
Yes % Juliet -> Lisa -> Harry
?- grandmother(mary, harry).
No % Mary is not Lisa's parent
?- grandmother(X, sandra).
X = juliet % Juliet -> Mary -> Sandra
;
X = bob % Bob -> Mary -> Sandra - Oops, need female! Check rule.
% Correct expected output should only be Juliet if rule checks gender.
% Let's assume the rule *will* check gender:
X = juliet
Yes
```

16. You are given a database of facts listing people and their birth dates. Dates are represented using the compound term `date(Day, Month, Year)`.

Database Facts:

```
born(jan, date(20, 3, 1977)).
```

```
born(jeroen, date(2, 2, 1992)).
born(joris, date(17, 3, 1995)).
born(jelle, date(1, 1, 2004)).
born(jesus, date(24, 12, 0)). % Assuming Year 0 is valid for the example
born(joop, date(30, 4, 1989)).
born(jannecke, date(17, 3, 1993)).
born(jaap, date(16, 11, 1995)).
```

Implement the following predicates based on this data:

i. `year(Year, Person)`: This predicate should find all people (`Person`) who were born in the specified `Year`. The predicate should be able to return multiple answers through backtracking if several people share the same birth year.
   - It needs to access the `born/2` facts.
   - It needs to extract the year component from the `date/3` term.
   - It needs to compare the extracted year with the input `Year`.

   Example:

   ```
   ?- year(1995, Person).
   Person = joris ;          % First match
   Person = jaap ;           % Second match on backtracking
   No                        % No more matches
   ```

ii. `older(Person1, Person2)`: This predicate should succeed if `Person1` is strictly older than `Person2`.
   - You first need to find the birth dates of both `Person1` and `Person2` using the `born/2` facts.
   - You will likely need to implement a helper predicate, for example, `before(Date1, Date2)`, which succeeds if `Date1` comes chronologically before `Date2`.
   - The `before/2` predicate must compare dates based on Year, then Month, then Day. Remember: someone born on an earlier date is older.
   - Assume dates are well-formed (e.g., no 31st of April). Use standard arithmetic comparisons (<).

   Examples:

   ```
   ?- older(jan, jeroen). % 1977 vs 1992
   Yes

   ?- older(joris, jannecke). % Mar 17 1995 vs Mar 17 1993 -> Jannecke is older
   No

   ?- older(joris, jaap). % Mar 1995 vs Nov 1995 -> Joris is older
   Yes

   ?- older(jannecke, X). % Find everyone younger than Jannecke (born 1993)
   X = joris ;
   X = jelle ;
   X = jaap ;
   No
   ```

17. In this exercise, you will perform arithmetic using a unary number system represented by Prolog lists.

    - Non-negative integers are represented as lists containing only the atom `x`.
    - The number 0 is represented by the empty list `[]`.
    - The number 1 is represented by `[x]`.
    - The number 2 is represented by `[x, x]`.
    - ...and so on. The number n is represented by a list of n `x`'s.
    - **Restriction:** You must implement the following predicates using **only list manipulation** (like the head/tail pattern '[H—T]', unification '=') and recursion. **Do not use** any standard Prolog arithmetic operators or predicates (e.g., `is`, `+`, `-`, `<`, `mod`, `length/2`).

    Implement the following predicates:

    i. `successor(UnaryNum, SuccessorNum)`: Given a unary number `UnaryNum`, unify `SuccessorNum` ↪ with the unary representation of the next integer (UnaryNum + 1).
       - Hint: How do you add one 'x' to the list representation?

       Examples:

       ```
       ?- successor([x, x, x], Result).  % Successor of 3
       Result = [x, x, x, x]             % Should be 4
       Yes
       ?- successor([], Result).          % Successor of 0
       Result = [x]                      % Should be 1
       Yes
       ```

    ii. `plus(Unary1, Unary2, Sum)`: Given two unary numbers `Unary1` and `Unary2`, unify `Sum` with the unary representation of their sum.
       - Hint: Think recursively. Adding 0 (`[]`) to any number `N` results in `N`. How can you reduce the problem of adding `[x|Tail1]` and `Unary2` to a simpler addition problem? This is similar to list concatenation.

       Examples:

       ```
       ?- plus([x, x], [x, x, x], Result). % 2 + 3
       Result = [x, x, x, x, x]            % Should be 5
       Yes
       ?- plus([], [x, x], Result).        % 0 + 2
       Result = [x, x]                     % Should be 2
       Yes
       ?- plus([x, x, x, x], [], Result).  % 4 + 0
       Result = [x, x, x, x]               % Should be 4
       Yes
       ?- plus([], [], Result).            % 0 + 0
       Result = []                         % Should be 0
       Yes
       ```

Part 4: Backtracking Control & Negation

18. Explain when `whoami(X)` succeeds given its definition:

```
whoami([]).
whoami([_, _ | Rest]) :- whoami(Rest).
```

19. Implement a Prolog predicate `remove_duplicates(ListIn, ListOut)`.

    - `ListIn` is the input list, which may contain duplicate elements.
    - `ListOut` should be unified with a list containing the same elements as `ListIn`, but with all duplicate occurrences removed.
    - The relative order of the *remaining* elements should be preserved. Specifically, if an element appears multiple times, keep only its *last* occurrence in the original list order. (This matches the behavior implied by the example).
    - **Crucially, your predicate must be deterministic.** When Prolog finds the correct `ListOut` ↪ , it should succeed exactly once. If the user attempts to force backtracking (e.g., by pressing the semicolon ';'), your predicate must not generate incorrect alternative lists containing elements that should have been removed.
    - To achieve this deterministic behavior and prevent incorrect alternatives, you **must use the cut operator (!)** appropriately within your rules. Consider where Prolog might make a choice that could lead to a wrong answer on backtracking if not cut.

    Examples:

```
?- remove_duplicates([a, b, a, c, d, d], List).
List = [b, a, c, d]  % Note: Keeps last 'a', last 'd'
Yes

?- remove_duplicates([a, b, a, c, d, d], List); true. % Test backtracking
List = [b, a, c, d]  % Should succeed only ONCE with this correct list
Yes

?- remove_duplicates([1, 2, 3, 2, 1], Result).
Result = [3, 2, 1]
Yes

?- remove_duplicates([], Result).
Result = []
Yes
```

20. Assume you have a database of facts defining who is married to whom. The database only contains facts of the form `married(Person1, Person2)`, indicating that `Person1` and `Person2` are married.

    Example Database:

```
married(peter, lucy).
married(paul, mary).
married(bob, juliet).
married(harry, geraldine).
```

Write a Prolog predicate `single(Person)` that succeeds if the given `Person` is **not** found as *either* the first argument *or* the second argument in any `married/2` fact within the database. If the person is found in any `married/2` fact, the predicate should fail.

You **must use the negation as failure operator (+)** to check for the absence of the relevant `married/2` facts. Remember that `+ Goal` succeeds if Prolog fails to prove `Goal`. You will need to check two conditions using negation.

Examples (using the database above):

```
?- single(mary).
No % Fails because married(paul, mary) exists

?- single(peter).
No % Fails because married(peter, lucy) exists

?- single(claudia). % Assuming claudia is not in any married fact
Yes % Succeeds because Prolog cannot prove married(claudia, _) or married(_, claudia)

?- single(X). % Ask Prolog to find *any* single person
No % Fails because, based *only* on the facts provided, everyone known could be married.
   % Prolog doesn't know about claudia unless she's explicitly mentioned somewhere else.
```

21. Implement Euclid's algorithm to compute the greatest common divisor (GCD) of two non-negative integers.

    - Your predicate should have the signature `gcd(A, B, GCD)`, where `A` and `B` are the input non-negative integers, and `GCD` is unified with their greatest common divisor.
    - Recall Euclid's algorithm:
      - If B is 0, then gcd(A, B) is A. (Base Case)
      - If B is greater than 0, then gcd(A, B) is the same as gcd(B, A mod B). (Recursive Step)
    - You will need to use arithmetic evaluation (`is`) and comparison (`=:=`, `>`).
    - **Crucially, your predicate must be efficient and deterministic.** Euclid's algorithm gives a unique result. Your Prolog implementation should succeed exactly once with the correct GCD. If backtracking is requested (e.g., by pressing ';'), it should not attempt to compute alternatives or enter unintended recursive loops.
    - To achieve this deterministic behavior, you **must use one or more cuts (!)** appropriately.

    Examples:

    ```
    ?- gcd(57, 27, X).
    X = 3
    Yes

    ?- gcd(27, 57, X). % Should handle order correctly via recursion
    X = 3
    Yes
    ```

```
?- gcd(56, 28, X).
X = 28
Yes

?- gcd(9, 0, X).
X = 9
Yes

?- gcd(0, 5, X). % Recursion should lead to gcd(5,0)
X = 5
Yes

?- gcd(57, 27, X); true. % Test backtracking - MUST succeed only once
X = 3
Yes
```

Part 5: Operators & Advanced Logic

22. Your task is to define Prolog operators to represent standard connectives in propositional logic. This allows you to write logical formulas more naturally within Prolog.

    - You need to define operators for:
      - Negation: Use the atom `neg`.
      - Conjunction: Use the atom `and`.
      - Disjunction: Use the atom `or`.
      - Implication: Use the atom `implies`.
    - Use the built-in predicate `op/3` to declare these. Remember the syntax: `op(Precedence, Type, Name)`. These declarations should typically be placed at the beginning of your Prolog file, preceded by `:-`.
    - **Operator Types:**
      - `neg` should be a prefix operator. Use type `fy` (associative prefix, allows e.g., `neg neg p`) or `fx` (non-associative prefix). `fy` is often suitable for negation.
      - `and`, `or`, `implies` should be infix operators.
    - **Precedence:** Operators need precedence values (lower numbers bind tighter). Define them according to the standard logical hierarchy: Negation binds tightest, then Conjunction, then Disjunction, then Implication binds weakest. Suggested values:
      - `neg`: 200
      - `and`: 400
      - `or`: 500
      - `implies`: 600
    - **Associativity:** Define the binary infix operators (`and`, `or`, `implies`) as left-associative using type `yfx`. This means an expression like `a and b and c` is parsed as `(a and b) and c`.

- **Testing:** You don't need to implement the evaluation of these formulas. To test if your operator definitions work correctly, you can define them using `:- op(...).` in your file or type the `op/3` goals directly into the interpreter. Then, try unifying a variable with a complex formula involving parentheses. Prolog's response will omit parentheses that are redundant according to your operator definitions, but will keep necessary ones. Compare this to the expected logical structure.

Example Tests (after defining operators):

```
?- Formula = (a implies ((b and c) and d)).
% Expected: Formula = a implies b and c and d
% (Parentheses around 'b and c' and 'd' are redundant if
% 'and' is left-associative and binds tighter than 'implies')
Yes

?- AnotherFormula = (neg (neg a)) or b.
% Expected: AnotherFormula = neg neg a or b
% (Parentheses around 'neg a' and 'neg neg a' are redundant
% if 'neg' is prefix 'fy')
Yes

?- ThirdFormula = (a or b) and c.
% Expected: ThirdFormula = (a or b) and c
%(Parentheses are necessary because 'and' binds tighter than 'or')
Yes
```

23. i. Define infix operators `plusplus` (for which you can use the atom `'++'`) and `timestimes` (atom `'**'`) with precedence 500 for `'++'` and 400 for `'**'` (lower binds tighter), both left-associative (`yfx`). Use `:- op(...)` declarations.

    ii. Write a predicate `eval_expr(Expression, Value)` that evaluates expressions composed of integers and your custom `'++'` and `'**'` operators. The evaluation rules are:

    - `eval_expr(Int, Int)` if Int is an integer.
    - `eval_expr(A ++ B, Val)` :- evaluate A to ValA, evaluate B to ValB, Val is ValA + ValB + 1.
    - `eval_expr(A ** B, Val)` :- evaluate A to ValA, evaluate B to ValB, Val is ValA * ValB * 2.

```
% After defining operators via :- op(...)
?- eval_expr(3 ++ 5 ** 2, Val).
% Should evaluate as 3 ++ (5 ** 2) due to precedence
% eval(5 ** 2) -> eval(5) * eval(2) * 2 = 5 * 2 * 2 = 20
% eval(3 ++ 20) -> eval(3) + eval(20) + 1 = 3 + 20 + 1 = 24
Val = 24
Yes
?- eval_expr(1 ** 2 ++ 3 ** 4, Val).
% Should evaluate as (1 ** 2) ++ (3 ** 4)
% eval(1 ** 2) -> 1 * 2 * 2 = 4
% eval(3 ** 4) -> 3 * 4 * 2 = 24
```

```
% eval(4 ++ 24) -> 4 + 24 + 1 = 29
Val = 29
Yes
```

This requires carefully handling the structure based on your operator definitions and applying the custom evaluation rules recursively.

24. Write a predicate `print_square(N, Char)` that prints an N x N square of the character `Char` to the console. Use recursion, `write/1`, and `nl/0`.

```
?- print_square(4, '*').
****
****
****
****
Yes
```